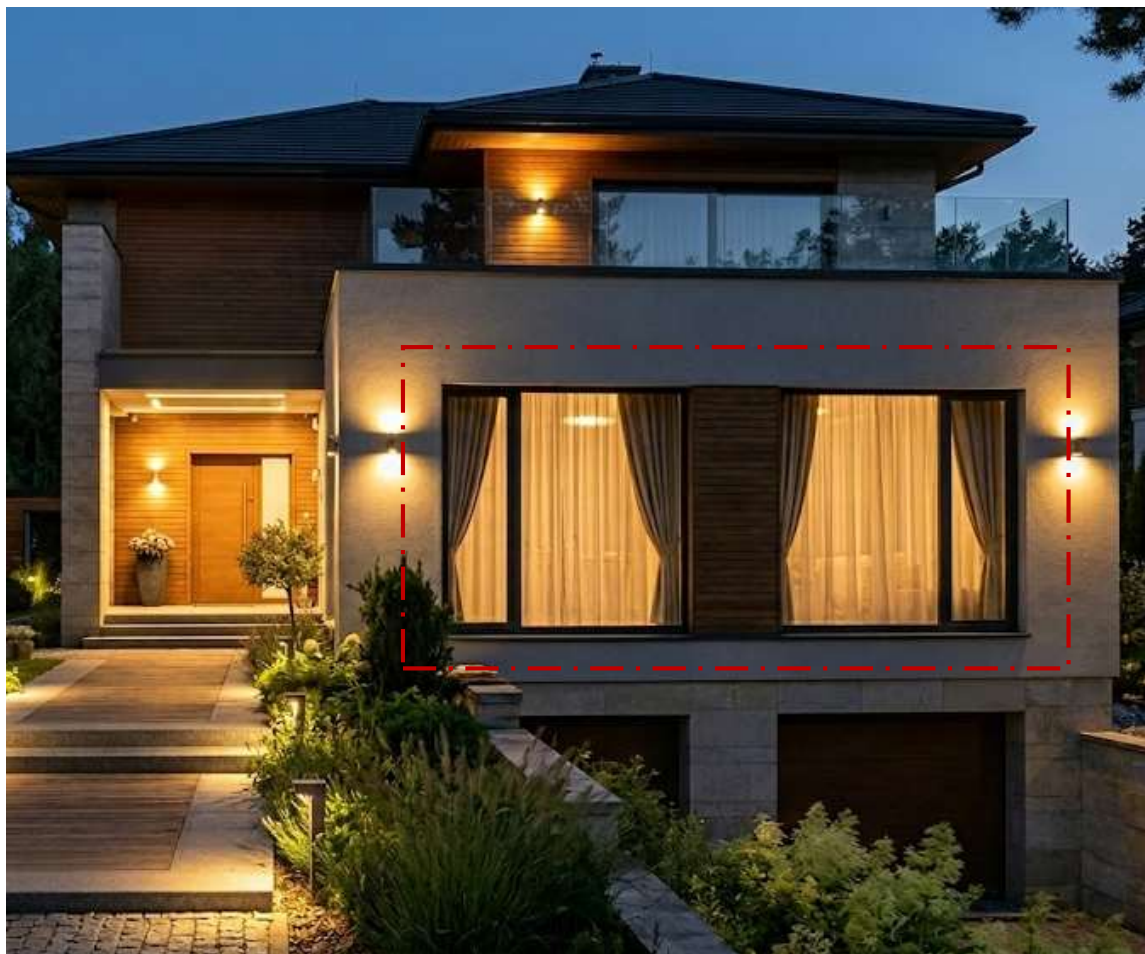


Présentation du projet

Objectifs de l'automatisme réalisé, sa mise en place et son architecture



Rideaux intelligents commandés par les capteurs LDR ou manuellement depuis Node-Red



Rideaux intelligents commandés par les capteurs LDR ou manuellement depuis Node-Red



Éclairage couloir intelligent commandé par les capteurs LDR et PIR , ou manuellement depuis Node-Red



Positionnement des capteurs PIR1, PIR2, LDR1 et LDR2

L'ensemble des capteurs est lié à une ESP32 à travers des fils électriques, l'ESP32 peut communiquer avec HiveMQ à travers le Wifi.

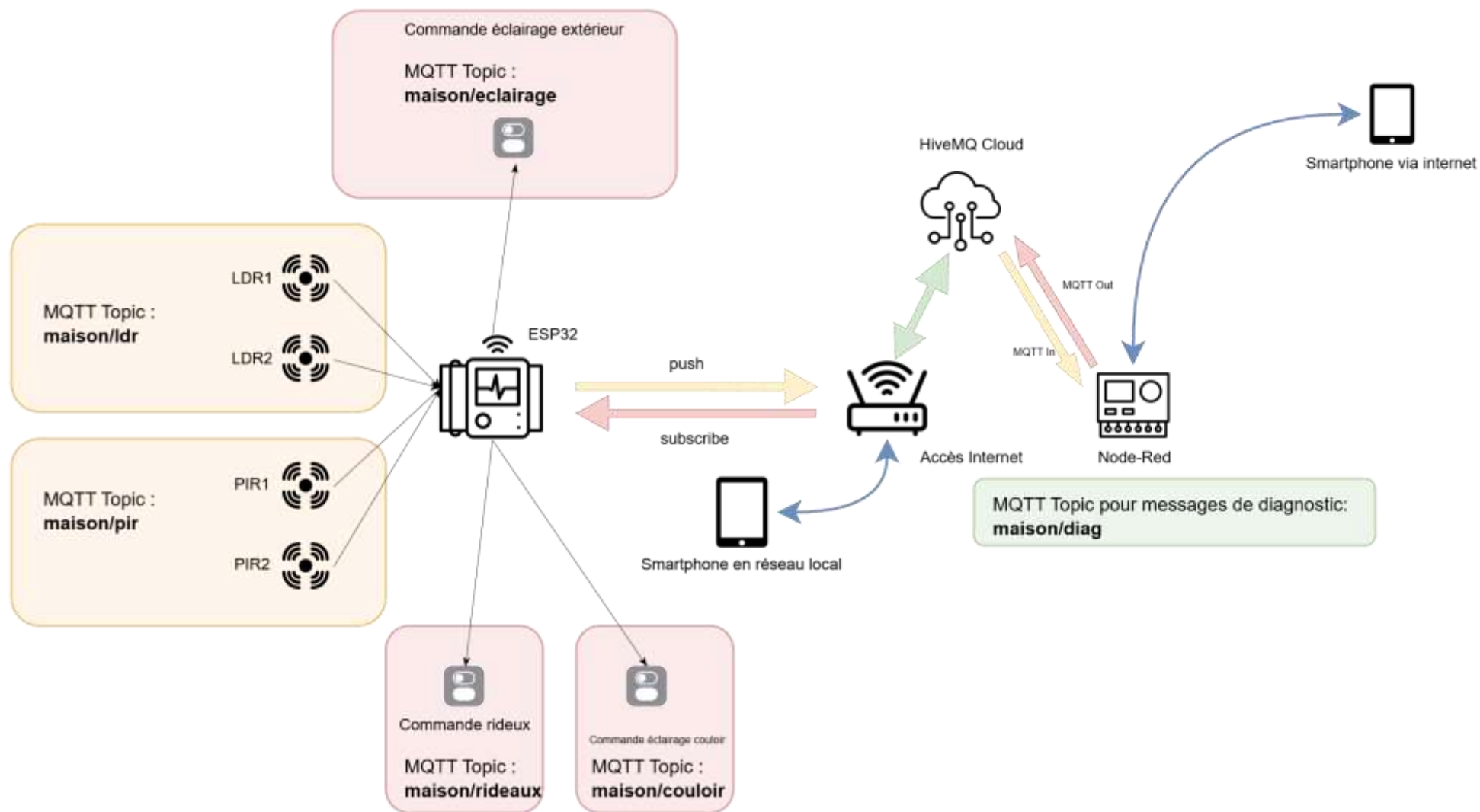


Schéma de connexion capteurs et actionneurs

Les différents échanges sont séparés à travers des topics MQTT centralisés via le broker HiveMQ Cloud

La logique est appliquée avec génération des commandes à travers Node-Res

Réalisation

Simulation ESP33, Capteurs et connexion au broker MQTT

HiveMQ

Q

Search in documentation

CtrlK

?

ORGANIZATIONS

USMBA ENSAF

Welcome to HiveMQ

Cloud Clusters

Licenses

Cloud Clusters

Create New Cluster

Free #1

▲ Running

Plan	Started	Port (TLS)	URL
Serverless	2026-04-12T16:31:12Z	8883	6d11058fc1944306afdf9c56d952159c.s1.eu.hivemq.cloud

Manage Cluster

Création de cluster gratuit sur HiveMQ Cloud

Création d'un cluster gratuit pour pouvoir échanger des messages MQTT via le cloud HiveMQ

HiveMQ

Q

Search in documentation

Ctrl K

?

ORGANIZATIONS

USMBA ENSAF

Welcome to HiveMQ

Cloud Clusters

Licenses

← Back

Free #1

Overview

Access Management

Web Client

Getting Started

Connection Details

Comprehensive details and statistics for your cluster

URL

6d11058fc1944306afdf9c56d952159c.s1.eu.hivemq.cloud

Port

8883

WebSocket Port

8884

TLS MQTT URL

6d11058fc1944306afdf9c56d952159c.s1.eu.hivemq.cloud:8883

TLS WebSocket URL

6d11058fc1944306afdf9c56d952159c.s1.eu.hivemq.cloud:8884/mqtt

Récupération des éléments de connexion

Chaque cluster dispose d'une URL unique avec accès possible par deux ports 8883 et 8884

← Back

Free #1

Overview

Access Management

Web Client •

Getting Started



Authentication

Credentials

Active

Hide

Currently you have not created any credentials. Fill out the following form to create an access credentials pair and limit access to your HiveMQ Cloud MQTT instance. To learn more check out our Security Fundamentals guide.

Username

ESP32-GSCSI-Douae

Permission

Publish and Subscribe

Password

.....



Confirm Password

.....



Save

Cancel

Création de compte avec permissions publication (envoi) et souscription (écoute et réception)

Il est nécessaire de créer une identité pour authentifier les différents composants de notre architecture

Authentication

Credentials Active

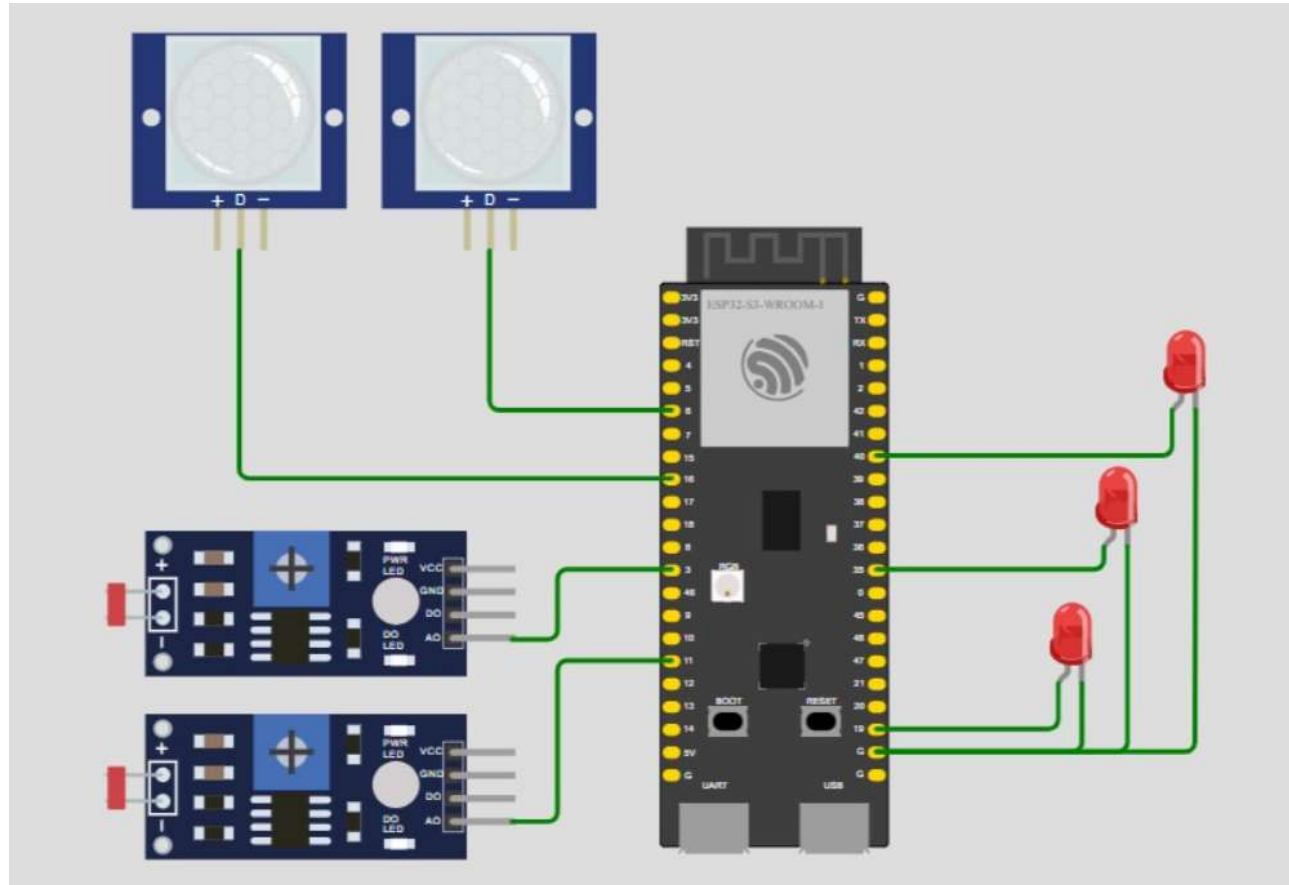
[Hide](#)

Currently you have not created any credentials. Fill out the following form to create an access credentials pair and limit access to your HiveMQ Cloud MQTT instance. To learn more check out our Security Fundamentals guide.

Name	Permission	Actions
ESP32-GSCSI-Douae	PUBLISH_SUBSCRIBE	
ESP32-GSCSI-Node-RED	PUBLISH_SUBSCRIBE	

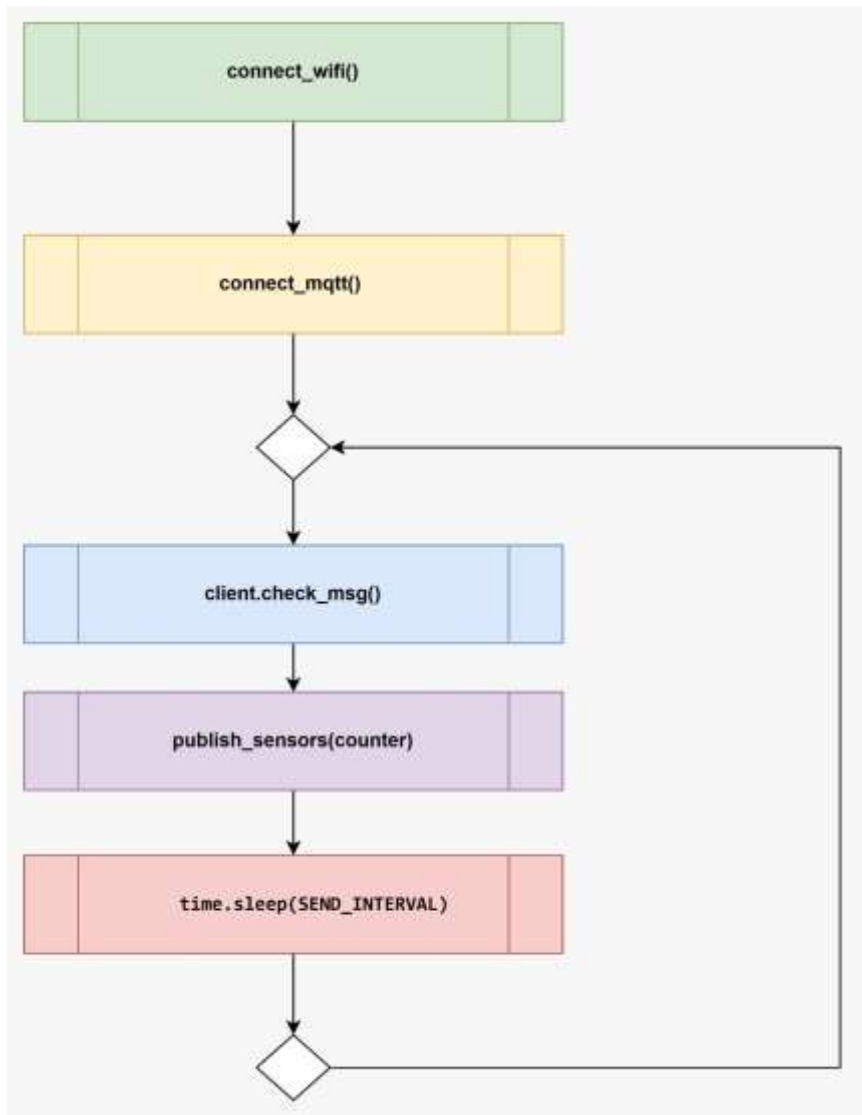
[Add Credentials](#)

Création de comptes pour l'ESP32 et pour Node-Red pour séparer les commandes de l'acquisition des capteurs



Branchement des capteurs et abstraction des actionneurs

Les capteurs utilisés (deux LDRs pour détection de la lumière ambiante, et deux PIRs pour détection de présence sont connectés directement à la carte ESP32 de l'acquisition.



Structure générale du code microPython

Le code microPython se déroule sur plusieurs étapes : initialisation des variables, déclaration des fonctions utiles, et puis appels à ces fonctions pour initiation de connexion MQTT avant d'entrer dans une boucle infinie d'écoute et de publication de messages sur différents topics.

```
SSID = "Wokwi-GUEST"
WIFI_PASSWORD = ""

# Informations relatives au broket MQTT ( cluster HiveMQ crée dans l'étape précédente )
MQTT_BROKER = "6d11058fc1944306afdf9c56d952159c.s1.eu.hivemq.cloud"
PORT = 8883

MQTT_USER = "ESP32-GSCSI-Douae"          # Nom d'utilisateur utilisé pour accès HiveMQ
MQTT_PASSWORD = "ESP32-GSCSI-Douae"      # Mot de passe utilisé pour accès HiveMQ

CLIENT_ID = "esp32_maison"
```

Déclaration des variables requis pour la connexion Wifi et MQTT

```
# Séparation des messages MQTT par topics
```

```
TOPIC_PIR = "maison/pir"
```

```
TOPIC_LDR = "maison/ldr"
```

```
TOPIC_DIAG = "maison/diag"
```

```
TOPIC_COULOIR = "maison/couloir"
```

```
TOPIC_ECLAIRAGE = "maison/eclairage"
```

```
TOPIC_RIDEAUX = "maison/rideaux"
```

```
SEND_INTERVAL = 1 # Interval d'attente pour éviter la saturation
```

Définition des topics utilisés pour l'envoi et la réception des commandes


```
# Initialisation des capteurs et des actionneurs
```

```
pir1 = Pin(6, Pin.IN)  
pir2 = Pin(16, Pin.IN)
```

```
ldr1 = ADC(Pin(3))  
ldr1.atten(ADC.ATTN_11DB)
```

```
ldr2 = ADC(Pin(11))  
ldr2.atten(ADC.ATTN_11DB)
```

```
led_couloir = Pin(40, Pin.OUT)  
led_eclairage = Pin(35, Pin.OUT)  
led Rideaux = Pin(19, Pin.OUT)
```

Déclaration des PINs utilisés pour les capteurs et les LEDs

```

# =====
# CALLBACK MQTT
# =====

def on_message(topic, msg):

    """
    Fonction exécutée lors de la réception d'un message MQTT
    Permet de piloter LEDs et rideaux
    """

    global etat_couloir, etat_eclairage, etat_rideaux

    topic = topic.decode()          # Décodage message MQTT reçu
    data = ujson.loads(msg)         # Transformation du message brut reçu

    print("Message reçu:", topic, data)

    # Condition en fonction du topic du message reçu, et de la valeur du paramètre de commande

    if topic == TOPIC_COULOIR:
        etat_couloir = data.get("status", 0)
        led_couloir.value(etat_couloir)

    elif topic == TOPIC_ECLAIRAGE:
        etat_eclairage = data.get("status", 0)
        led_eclairage.value(etat_eclairage)

    elif topic == TOPIC_RIDEAUX:
        etat_rideaux = data.get("status", 0)
        led_rideaux.value(etat_rideaux)

```

Définition de la fonction d'écoute et récupération de la commande à travers les messages MQTT

```

def publish_sensors(counter):

    # Construction du message MQTT PIR
    pir_data = {
        "id": counter,
        "pir1": pir1.value(),
        "pir2": pir2.value()
    }

    # Construction du message MQTT LDR
    ldr_data = {
        "id": counter,
        "ldr1": ldr1.read(),
        "ldr2": ldr2.read()
    }

    # Construction du message MQTT de diagnostic global
    diag_data = {
        "id": counter,
        "pir": pir_data,
        "ldr": ldr_data,
        "timestamp": time.time()
    }

    # Publication des messages MQTT vers les topics dédiés
    client.publish(TOPIC_PIR, ujson.dumps(pir_data))
    client.publish(TOPIC_LDR, ujson.dumps(ldr_data))
    client.publish(TOPIC_DIAG, ujson.dumps(diag_data))
    print("Capteurs envoyés")

```

Définition de la fonction de création et de publication des messages MQTT

```

# =====
# CONNEXION MQTT
# =====

def connect_mqtt():
    global client
    # Utilisation des variables définies pour l'authentification

    client = MQTTClient(
        CLIENT_ID,
        MQTT_BROKER,
        port=PORT,
        user=MQTT_USER,
        password=MQTT_PASSWORD,
        ssl=True,
        ssl_params={"server_hostname": MQTT_BROKER}
    )
    client.set_callback(on_message)
    client.connect()
    # Abonnement aux topics des actionneurs
    client.subscribe(TOPIC_COULOIR)
    client.subscribe(TOPIC_ECLAIRAGE)
    client.subscribe(TOPIC_RIDEAUX)

    print("MQTT connecté et abonné aux commandes")

```

Définition de la fonction de connexion au broker MQTT et de souscription au topics commandes


```
✓ def connect_wifi():          # Connexion Wifi simulé sur Wokwi

    wifi = network.WLAN(network.STA_IF)
    wifi.active(True)
    wifi.connect(SSID, WIFI_PASSWORD)

    print("Connexion WiFi...")

    while not wifi.isconnected():
        |   time.sleep(1)

    print("WiFi connecté:", wifi.ifconfig()[0])
```

Définition de la fonction de connexion Wifi

```
connect_wifi()      # Connexion Wifi
connect_mqtt()      # Connexion au broker MQTT
counter = 0         # Initialisation compteur pour identification des messages envoyés
```

```
while True:
```

```
    # Vérifie réception commandes MQTT (vidage complet de la file)
```

```
    for _ in range(10):
```

```
        | client.check_msg()
```

```
    # Applique le dernier état connu aux actionneurs
```

```
    led_couloir.value(etat_couloir)
```

```
    led_eclairage.value(etat_eclairage)
```

```
    led Rideaux.value(etat Rideaux)
```

```
    # Publication des données capteurs
```

```
    publish_sensors(counter)
```

```
    counter += 1
```

```
    gc.collect()
```

```
    # Attente d'un intervalle pour éviter la saturation
```

```
    time.sleep(SEND_INTERVAL)
```

Exécution des fonctions déclarés pour initialisation et en boucle infinie

✓ The WebClient is connected

Connection Settings

Connect to your HiveMQ Cloud Cluster with your credentials. Or, connect instantly with autogenerated credentials.

Username Password
ESP32-GSCSI-Douae

Disconnect

Topic Subscriptions 3

Unsubscribe from all topics

Subscribe to topics to receive messages from the HiveMQ cluster. You can also set the Quality of Service (QoS) for each topic. The higher the QoS, the more reliable the message delivery is. You can always subscribe to the (*) wildcard to receive all messages. Please note that the messages from internal probe topics are not displayed here.

Topic	QoS	Actions
 maison/pir	QoS: 0	
 maison/ldr	QoS: 0	
 maison/diag	QoS: 0	

Topic Name Subscribe

Messages 257

```
{ "ldr1": 3769, "id": 79, "ldr2": 1001 }
237 Topic: maison/pir QoS: 0
{ "pir2": 0, "id": 79, "pir1": 0 }
238 Topic: maison/diag QoS: 0
{ "id": 79, "timestamp": 90, "pir": { "pir2": 0, "id": 79, "pir1": 0 }, "ldr": { "ldr1": 3769, "id": 79, "ldr2": 1001 } }
239 Topic: maison/ldr QoS: 0
{ "ldr1": 3769, "id": 80, "ldr2": 1001 }
240 Topic: maison/pir QoS: 0
{ "pir2": 0, "id": 80, "pir1": 0 }
241 Topic: maison/diag QoS: 0
{ "id": 80, "timestamp": 91, "pir": { "pir2": 0, "id": 80, "pir1": 0 }, "ldr": { "ldr1": 3769, "id": 80, "ldr2": 1001 } }
242 Topic: maison/pir QoS: 0
{ "pir2": 0, "id": 81, "pir1": 0 }
243 Topic: maison/ldr QoS: 0
{ "ldr1": 3769, "id": 81, "ldr2": 1001 }
244 Topic: maison/diag QoS: 0
{ "id": 81, "timestamp": 92, "pir": { "pir2": 0, "id": 81, "pir1": 0 }, "ldr": { "ldr1": 3769, "id": 81, "ldr2": 1001 } }
245 Topic: maison/pir QoS: 0
{ "pir2": 0, "id": 82, "pir1": 0 }
246 Topic: maison/ldr QoS: 0
{ "ldr1": 3769, "id": 82, "ldr2": 1001 }
247 Topic: maison/diag QoS: 0
{ "id": 82, "timestamp": 94, "pir": { "pir2": 0, "id": 82, "pir1": 0 }, "ldr": { "ldr1": 3769, "id": 82, "ldr2": 1001 } }
248 Topic: maison/ldr QoS: 0
{ "ldr1": 3769, "id": 83, "ldr2": 1001 }
249 Topic: maison/pir QoS: 0
{ "pir2": 0, "id": 83, "pir1": 0 }
```

Vérification sur le WebClient au niveau de la console HiveMQ on reçoit bien les messages MQTT

Send Message 64

If you cannot see any messages, make sure you are subscribed to the correct topics. You can always subscribe to the (#) wildcard to receive all messages.

Message

{"status": 1}

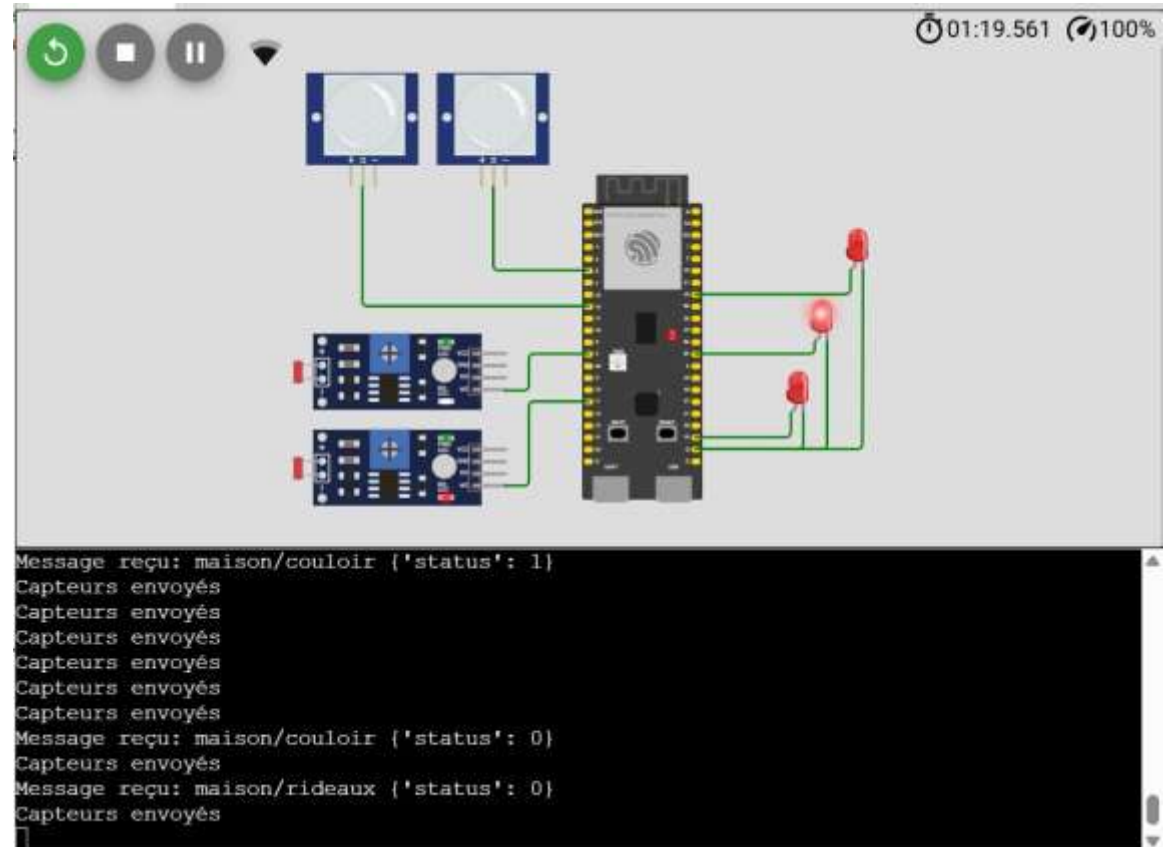
Topic

maison/eclairage

QoS

QoS: 0

Send Message



Test d'envoi de message dans une topic commande et test de réception de la commande

Contrôle avancé avec Node-Red

Contrôle et création de tableau de bord avec Node-Red

Edit mqtt in node > **Edit mqtt-broker node**

Delete Cancel **Update**

Properties

Name HiveMQ

Connection Security Messages

Server 6d11058fc1944306afd9c56d952159c.s1.eu.hi **Port** 8883

☒ Connect automatically

☒ Use TLS TLS configuration  

Protocol MQTT V3.1.1

Client ID Leave blank for auto generated

Keep Alive 60

Session ☒ Use clean session

Edit mqtt in node > **Edit mqtt-broker node**

Delete Cancel **Update**

Properties

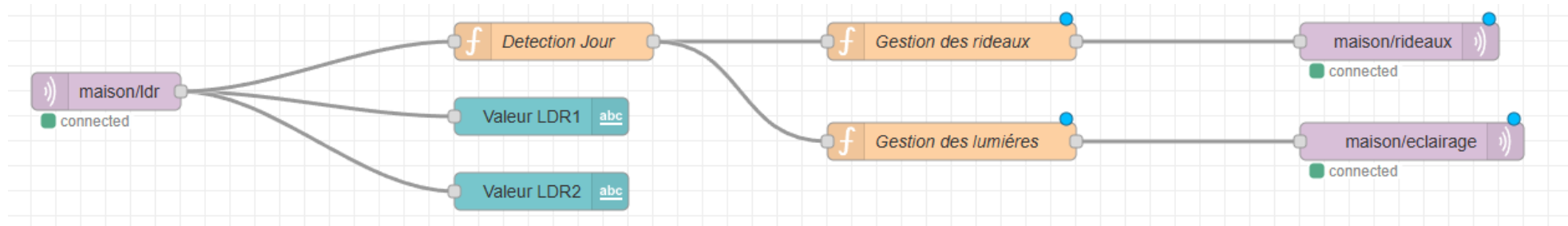
Name HiveMQ

Connection **Security** Messages

Username ESP32-GSCSI-Node-RED

Password

Connexion et authentification de Node Red vers le broker MQTT



Edit mqtt in node

Delete Cancel Done

Properties

Server: 6d11058fc1944306afdf9c56d95215

Action: Subscribe to single topic

Topic: maison/ldr

QoS: 2

Output: a parsed JSON object

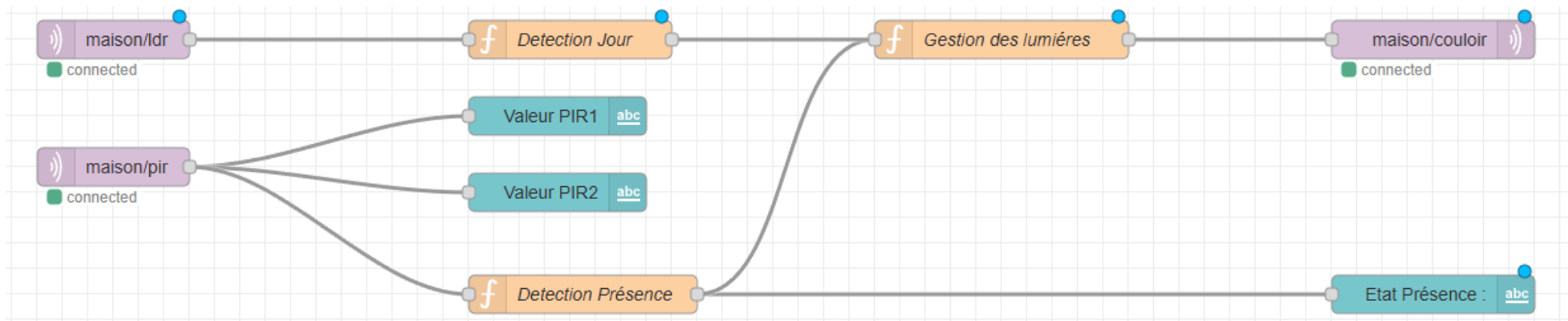
Name: Name

Name: Detection Jour

Setup On Start On Message On Stop

```
1 if (msg.payload.ldr1 < 1800 && msg.payload.ldr2 < 1800) {
2   msg.payload = "jour";
3 } else {
4   msg.payload = "nuit";
5 }
6
7 return msg;
```

Souscription à la topic maison/ldr pour récupération de la valeur des capteurs LDR et définition d'une fonction de détection jour et nuit par seuil d'éclairage



Edit mqtt in node

Delete Cancel Done

Properties

Server 6d11058fc1944306afdf9c56d9521

Action Subscribe to single topic

Topic maison/pir

QoS 2

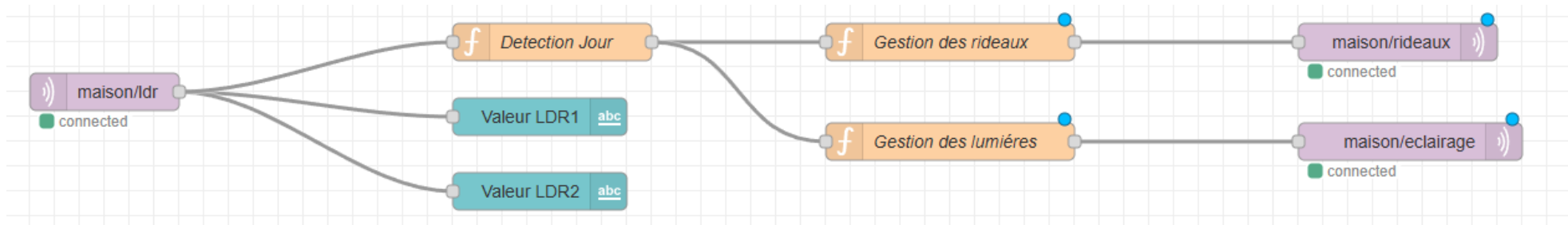
Output a parsed JSON object

Name Detection Présence

Setup On Start On Message On Stop

```
1 if (msg.payload.pir1 === 1 || msg.payload.pir2 === 1) {
2   msg.payload = "presence";
3   return msg;
4 }
5 else
6 {
7   msg.payload = "-";
8   return msg;
9 }
10 return null;
```

Souscription à la topic maison/pir pour récupération de la valeur des capteurs PIR et définition d'une fonction de détection de présence



Name: Gestion des rideaux

Setup On Start On Message On Stop

```

1  if (msg.payload === "jour") {
2    msg.topic = "maison/rideaux";
3    msg.payload = { status: 1 };
4  } else {
5    msg.topic = "maison/rideaux";
6    msg.payload = { status: 0 };
7  }
8
9  return msg;

```

Name: Gestion des lumières

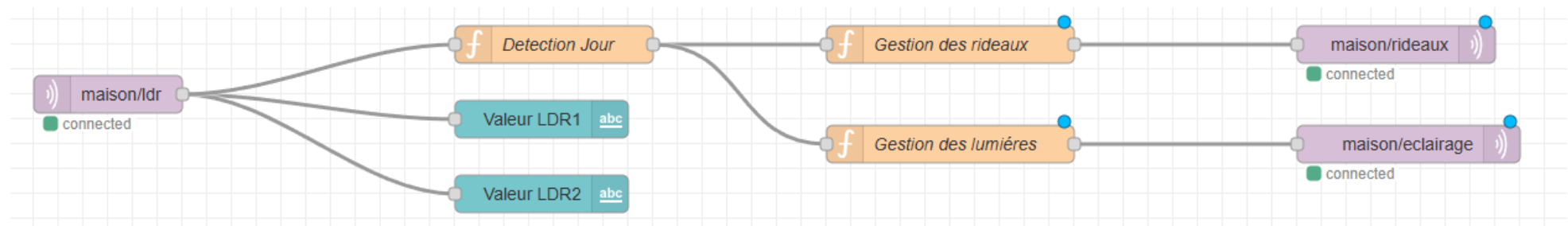
Setup On Start On Message On Stop

```

1  if (msg.payload === "nuit") {
2    msg.topic = "maison/eclairage";
3    msg.payload = { status: 1 };
4    return msg;
5  } else{
6    msg.topic = "maison/eclairage";
7    msg.payload = { status: 0 };
8    return msg;
9  }
10 }
11

```

Définition de fonctions pour génération de message MQTT pour la gestion des rideaux et de la lumière



Delete

Cancel

Done

⚙ Properties

⚙

📄

🖼

🌐 Server

6d11058fc1944306afdf9c56d952159

✎

+

📄 Topic

maison/rideaux

🔊 QoS

▼

🔄 Retain

▼

🔖 Name

Name

Tip: Leave topic, qos or retain blank if you want to set them via msg properties.

Delete

Cancel

Done

⚙ Properties

⚙

📄

🖼

🌐 Server

6d11058fc1944306afdf9c56d952159

✎

+

📄 Topic

maison/eclairage

🔊 QoS

▼

🔄 Retain

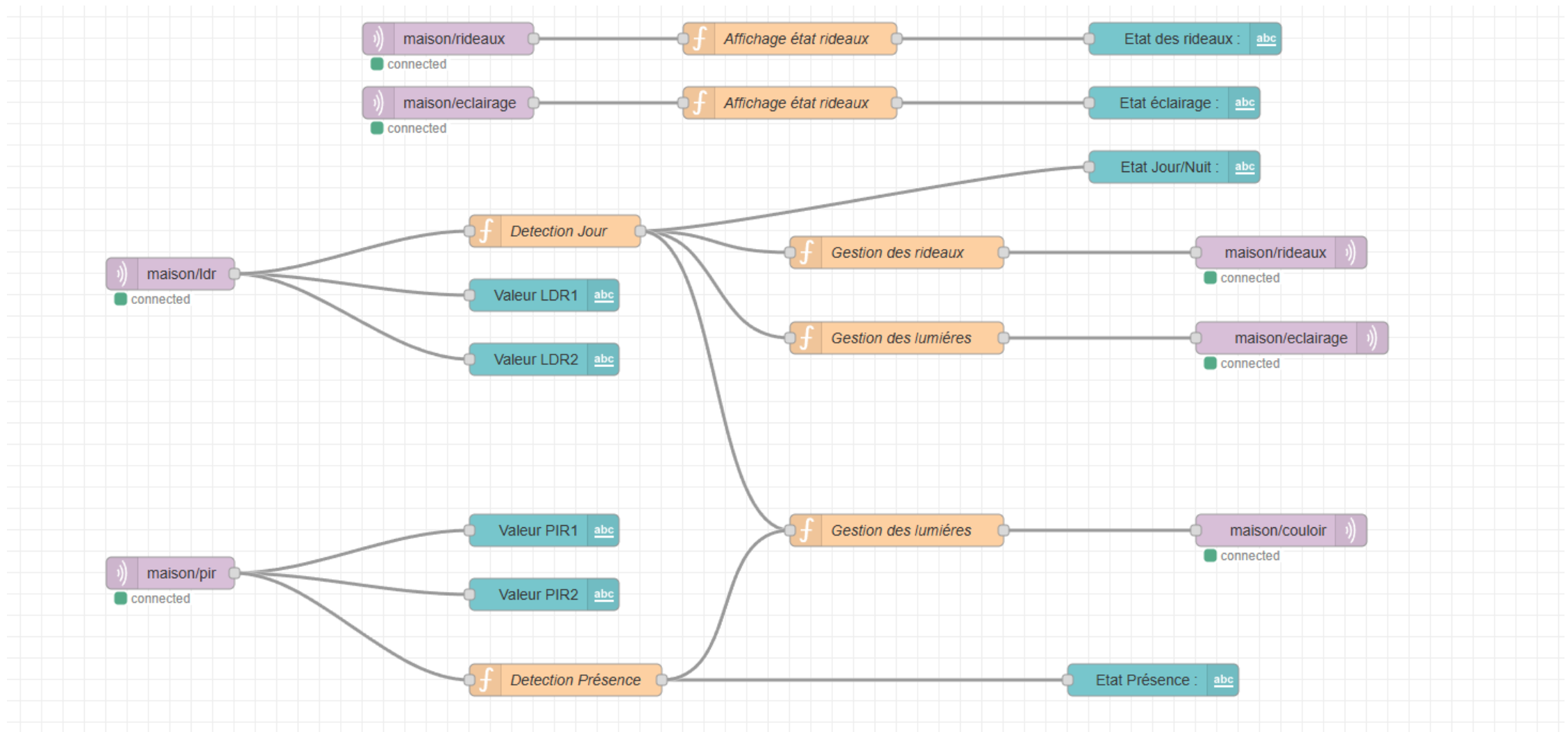
▼

🔖 Name

Name

Tip: Leave topic, qos or retain blank if you want to set them via msg properties.

Diffusion du message MQTT sur les topics maison/rideaux et maison/eclairage



Flux utilisé pour le projet : acquisition des valeurs des capteurs, logique et diffusion des commandes générés par MQTT

Node-RED Dashboard

127.0.0.1:1880/ui/#!/0?socketid=cTyaGPlex-CBAB2RAAAB

Scolaire

GSCSI

Valeurs Capteurs		Status	
Valeur LDR2	488	Etat des rideaux :	1. Ouverts
Valeur PIR1	0	Etat éclairage :	0. Eteints
Valeur PIR2	0	Etat Jour/Nuit :	jour
Valeur LDR1	344	Etat Présence :	-
		Etat du couloir :	0. Eteint

Accès à l'interface graphique pour diagnostic et affichage des états des capteurs et des commandes

WOKWI SAVE SHARE Docs 0

Simulation 02:39.118 99%

```

Message reçu: maison/eclairage ('status': 0)
Message reçu: maison/rideaux ('status': 1)
Message reçu: maison/couloir ('status': 0)
Message reçu: maison/couloir ('status': 0)
Capteurs envoyés

```

GSCSI

Valeurs Capteurs

Valeur LDR2	488
Valeur PIR1	0
Valeur PIR2	0
Valeur LDR1	344

Status

Etat des rideaux :	1. Ouverts
Etat éclairage :	0. Eteints
Etat Jour/Nuit :	jour
Etat Présence :	-
Etat du couloir :	0. Eteint

Tests avec l'état : Lumière du jour avec absence de mouvement

WOKWI SAVE SHARE Docs 0

Simulation 02:55.236 100%

```

Message reçu: maison/rideaux {'status': 0}
Message reçu: maison/couloir {'status': 0}
Message reçu: maison/eclairage {'status': 1}
Message reçu: maison/couloir {'status': 0}
Capteurs envoyés
Message reçu: maison/eclairage {'status': 1}

```

GSCSI

Valeurs Capteurs

Valeur LDR2	3499
Valeur PIR1	0
Valeur PIR2	0
Valeur LDR1	3116

Status

Etat des rideaux :	0. Fermés
Etat éclairage :	1. Allumés
Etat Jour/Nuit :	nuir
Etat Présence :	-
Etat du couloir :	0. Eteint

Tests avec l'état : Lumière faible (nuit) sans mouvement

WOKWI SAVE SHARE Docs 0

Simulation 03:12.769 99%

PIR Motion Sensor ×

Simulate motion

```

Message reçu: maison/couloir {'status': 1}
Message reçu: maison/eclairage {'status': 1}
Message reçu: maison/couloir {'status': 1}
Capteurs envoyés
Message reçu: maison/rideaux {'status': 0}
Message reçu: maison/couloir {'status': 1}
  
```

GSCSI

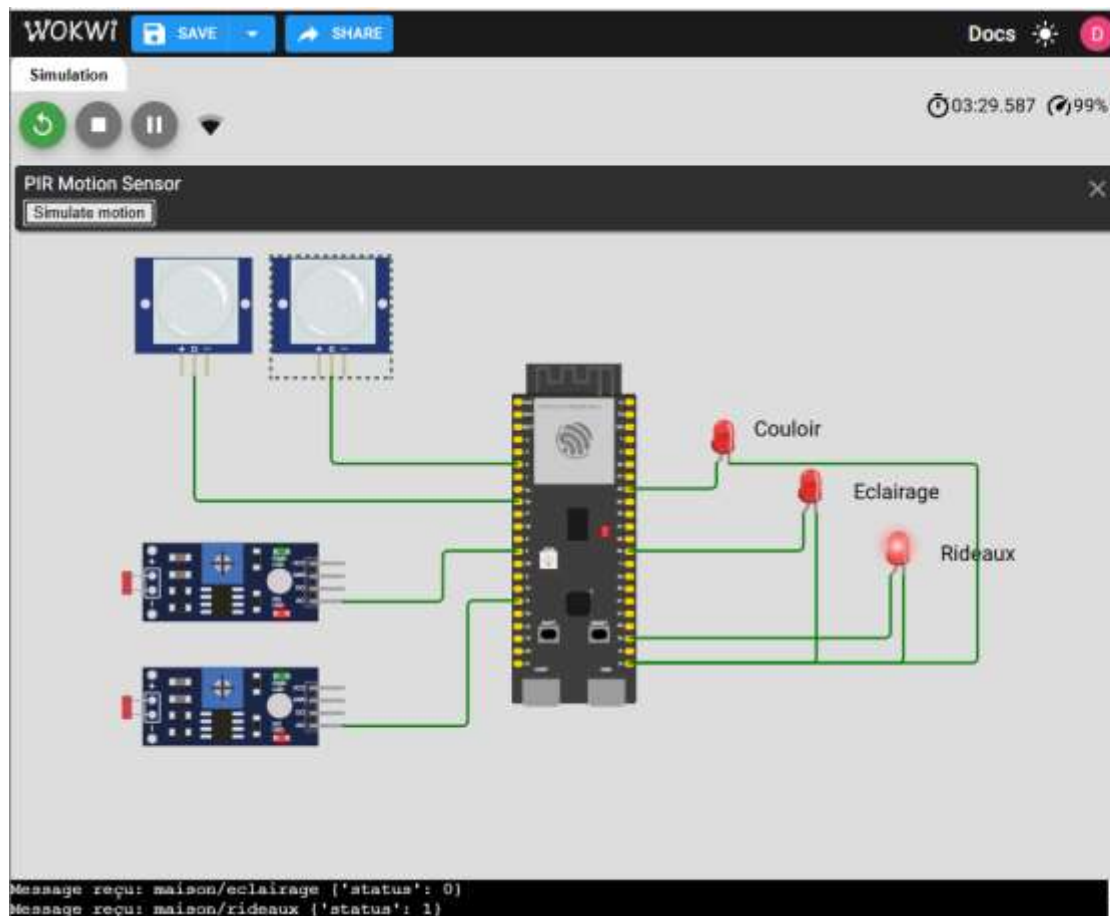
Valeurs Capteurs

Valeur LDR2	3499
Valeur PIR1	1
Valeur PIR2	0
Valeur LDR1	3116

Status

Etat des rideaux :	0. Fermés
Etat éclairage :	1. Allumés
Etat Jour/Nuit :	nuît
Etat Présence :	presence
Etat du couloir :	1. Allumé

Tests avec l'état : Lumière faible (nuit) avec mouvement



GSCSI

Valeurs Capteurs

Valeur LDR2	488
Valeur PIR1	1
Valeur PIR2	0
Valeur LDR1	698

Status

Etat des rideaux :	1. Ouverts
Etat éclairage :	0. Eteints
Etat Jour/Nuit :	jour
Etat Présence :	presence
Etat du couloir :	0. Eteint

Tests avec l'état : Lumière du jour avec mouvement detecté